

CARD 1 · EVERY LINE

1. Kind?

declaration · assignment · bare expression · control · type-only (runs nothing → stop)

2. Source?

literal · call · property read · operator · `await` · destructure

3. Prim / Obj?

string number boolean undefined null → by value.
Else → by address.

4. Ticket?

`async` · `: Promise<` · `fetch / db / .then / new Promise` · body returns one → needs `await` / `return / .then / Promise.all`

5. Nothing?

`?` prop · `find` · `map.get` · `arr[i]` · `headers/query/params` · fall-off-end · `?.` → is next · guarded? (`!x` also eats 0 "" false)

6. Who else?

new address (literal, spread, map/filter, clone) or alias (= a, param, find, get, return this.x)?
mutated? who holds it?

7. Kept?

assigned · returned · awaited · passed · used in if
→ else dropped: dead line or fire-and-forget

NOTE: label holds value · own / shared with ____ · nothing? ____ · ticket / value

LOOKS WRONG, IS FINE

- `return await` inside try · `!` right after `has` · `readonly` param · assigning to `never` · `WeakSet` default param · `let t` before executor assigns it · shared counter with no await between read and `++` · `x == null`

LOOKS FINE, IS WRONG

- `=` in a condition · `sort()` no comparator / as a copy · `||` on a 0 · `NaN` past a range check · `for...` in on array · `forEach(async)` · `try { f() }` no await · `res.json()` no return · `exp` secs vs `Date.now()` ms · `Object.assign(DEFAULTS,...)` · `slice(-0)` · `new Date(bad)` is truthy · `if (arr)`

Say it out loud. Plug in a value. Write the note before the next line.

CARD 2 · THE REVIEW

READING ORDER (WHOLE PROGRAM)

1. **Clauses** — rewrite the requirement as numbered facts
2. **Types** — every `?`, `| null`, `any`
3. **State** — anything outside a function: shared, races, leaks
4. **Signatures** — params, return type, `async`?
5. **Bodies** — entry points, card per line
6. **Helpers** — at the moment they're called, with the caller's values
7. **Lifecycle** — timers, listeners: cleared? unref'd? removed?
8. **Clauses again** — point each at a line, or write "not implemented"

CHECK ORDER (SEVERITY)

requirement → `=/==` → boundary (limit, ± 1) → `await` → nothing → input (undefined "" 0 -1 NaN "abc" huge) → mutation / who else → kept → errors (try+await, res.ok, swallow) → trust (SQL, URL, auth order, id from token) → race (read-await-write) → growth (no eviction, timer) → return paths → efficiency

RANK → VERDICT

a	violates a clause / crashes on real input	1-2 · reject / changes
b-d	edge input · security · leak or race	2-3 · request changes
e-g	design limit · efficiency · style	4-5 · approve w/ notes

Finding = line → consequence → fix, one sentence each. Two real positives always. "Possible, not confirmed: ..." beats a wrong certainty. Never default to 3.

LOOM (6 MIN, CAMERA ON)

0:30	"The requirement is A, B, C. I'll check each."
1:30	Walk top → bottom, one sentence per block, flag what to watch
3:00	Findings in rank: point at line · read it · plug in a value · "so..." · fix
0:30	Two positives, each with "because"
0:30	"Confidence N because _____. Verdict. If only X were wrong I'd say _____."
0:10	"I used AI to ____; the rest is mine."

Every finding has a "so". Cursor on every line you name. No silence > 3 s.